P R O D U C T R E Q U I R E M E N T S
D O C U M E N T

LensLife

iPhone Photography Portfolio & Showcase

Version 1.0
March 2026
Status: Draft

1. Product Overview

1.1 Vision

LensLife is a personal photography portfolio website designed to showcase iPhone photography. The site will serve as a curated, visually-driven experience that highlights the photographer's work across categorized galleries, supported by an about page and a contact form. The design will be created in Figma and implemented as a custom full-stack application.

1.2 Project Metadata

Attribute	Value
Project Name	LensLife — iPhone Photography Portfolio
Type	Personal Portfolio / Showcase
Owner	You (sole developer & photographer)
Design Tool	Figma
Target Launch	TBD (phased approach)
Methodology	Iterative sprints, self-managed

1.3 Goals & Non-Goals

Goals

- Build a polished, responsive photography portfolio that showcases iPhone photography at its best
- Gain hands-on experience with Docker for containerized development and deployment
- Implement a clean, maintainable full-stack architecture using Angular, Spring Boot, and PostgreSQL
- Create an admin interface for managing gallery content without code changes
- Deliver fast page loads with optimized image serving (lazy loading, responsive images, WebP)

Non-Goals (v1)

- E-commerce / print sales
- User accounts or community features
- Blog or journal section (potential v2 feature)
- SEO-heavy content marketing strategy

2. Technology Stack

2.1 Architecture Overview

The application follows a decoupled architecture with Angular as a standalone SPA communicating with a Spring Boot REST API backed by PostgreSQL. All services are containerized with Docker and orchestrated via Docker Compose for both local development and production deployment.

Layer	Technology	Version	Purpose
Frontend	Angular	19.x	SPA with standalone components, Angular Router, HttpClient
Backend	Spring Boot	3.4.x	REST API, image processing, admin auth
Database	PostgreSQL	16.x	Photo metadata, categories, contact form submissions
Containerization	Docker + Compose	Latest	Dev/prod environment parity, learning objective
Image Storage	Local filesystem (v1)	—	Mounted Docker volume; cloud (S3) migration path in v2
Reverse Proxy	Nginx	Latest	Serve Angular static files, proxy API, SSL termination

2.2 Architecture Diagram (Text)

```
Browser (Client)
    |
    ▼
[Nginx Container]  ← Serves Angular static build + proxies /api/*
    |
    ▼
[Spring Boot Container]  ← REST API on port 8080
```

```
    |                |
    ▼                ▼
[PostgreSQL]     [Image Volume]
```

2.3 Docker Strategy

Since learning Docker is a key objective, the project will use Docker from day one. Here's the container strategy:

Container	Base Image	Responsibility
nginx	nginx:alpine	Reverse proxy, serve Angular build, SSL
api	eclipse-temurin:21-jre-alpine	Spring Boot JAR, REST endpoints, image processing
db	postgres:16-alpine	PostgreSQL with mounted data volume

Docker Compose will define three profiles: dev (with hot reload and debug ports), staging (production-like locally), and prod (optimized images, resource limits). A shared .env file manages environment-specific configuration.

3. Feature Specifications

3.1 Photo Gallery System

The core feature. Users browse photos organized into categories/albums. The gallery should feel fast and immersive, prioritizing the photography itself.

Public Gallery

- Masonry or grid layout (driven by Figma design) with smooth transitions
- Category/album filtering with animated route transitions
- Lightbox view with swipe navigation, EXIF data overlay (camera model, settings)
- Lazy loading with blur-up placeholder technique (tiny base64 thumbnail → full image)
- Responsive images via srcset with WebP format, falling back to JPEG
- Optional photo title and short description

Admin Gallery Management

- Protected admin route (simple JWT auth, single-user)
- Drag-and-drop photo upload with progress indicators
- Auto-generate multiple sizes on upload (thumbnail 400px, medium 1200px, full 2400px)
- Auto-extract and store EXIF metadata from uploaded photos
- CRUD for categories/albums, drag-to-reorder photos within albums
- Bulk upload support

3.2 About Page

A single-page section with the photographer's bio, a profile/hero image, and optional links to social profiles. Content is admin-editable via a simple rich text field stored in the database.

3.3 Contact Form

A simple form with fields for name, email, subject, and message. Submissions are stored in PostgreSQL and optionally forwarded via email (SMTP integration). Includes honeypot spam protection and rate limiting on the API endpoint.

Field	Type	Validation
Name	Text input	Required, 2–100 characters
Email	Email input	Required, valid email format
Subject	Text input	Required, 5–200 characters
Message	Textarea	Required, 10–5000 characters
Honeypot	Hidden field	Must be empty (bot trap)

4. Data Model

4.1 Entity Relationship Overview

The database schema is intentionally simple for v1. Four core tables handle all functionality, with room to extend for blog posts, tags, or user accounts in future versions.

4.2 Tables

categories

Column	Type	Constraints	Notes
id	BIGSERIAL	PK	Auto-increment
name	VARCHAR(100)	NOT NULL, UNIQUE	Display name
slug	VARCHAR(100)	NOT NULL, UNIQUE	URL-friendly identifier
description	TEXT	NULLABLE	Optional description
sort_order	INTEGER	NOT NULL DEFAULT 0	Display ordering
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	Auto-updated via trigger

photos

Column	Type	Constraints	Notes
id	BIGSERIAL	PK	Auto-increment
category_id	BIGINT	FK → categories(id)	Album association
title	VARCHAR(200)	NULLABLE	Optional title
description	TEXT	NULLABLE	Optional caption
filename_original	VARCHAR(500)	NOT NULL	Original upload filename
filename_thumb	VARCHAR(500)	NOT NULL	400px thumbnail path
filename_medium	VARCHAR(500)	NOT NULL	1200px medium path
filename_full	VARCHAR(500)	NOT NULL	2400px full-size path

Column	Type	Constraints	Notes
placeholder_base64	TEXT	NULLABLE	Tiny blur-up placeholder
width	INTEGER	NOT NULL	Original dimensions
height	INTEGER	NOT NULL	Original dimensions
exif_data	JSONB	NULLABLE	Camera model, ISO, aperture, etc.
sort_order	INTEGER	NOT NULL DEFAULT 0	Within category
is_featured	BOOLEAN	NOT NULL DEFAULT FALSE	Homepage highlight
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	

contact_submissions

Column	Type	Constraints	Notes
id	BIGSERIAL	PK	Auto-increment
name	VARCHAR(100)	NOT NULL	Sender name
email	VARCHAR(255)	NOT NULL	Sender email
subject	VARCHAR(200)	NOT NULL	Message subject
message	TEXT	NOT NULL	Message body
is_read	BOOLEAN	NOT NULL DEFAULT FALSE	Admin read status
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	

site_settings

Column	Type	Constraints	Notes
key	VARCHAR(100)	PK	Setting identifier
value	TEXT	NOT NULL	JSON or plain text value
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	

Used for about page content, site title, social links, and other editable config.

5. API Design

5.1 REST Endpoints

All endpoints are prefixed with /api/v1. Admin endpoints require a valid JWT Bearer token.

Public Endpoints

Method	Endpoint	Description
GET	/categories	List all categories (sorted)
GET	/categories/{slug}	Get category with photos
GET	/photos	List photos (filterable by category, featured)
GET	/photos/{id}	Single photo with EXIF data
GET	/images/{size}/{filename}	Serve optimized image (thumb/medium/full)
POST	/contact	Submit contact form (rate-limited)
GET	/settings/{key}	Get public site setting

Admin Endpoints

Method	Endpoint	Description
POST	/auth/login	Authenticate, receive JWT
POST	/admin/photos	Upload photo(s) with auto-processing
PUT	/admin/photos/{id}	Update photo metadata
DELETE	/admin/photos/{id}	Delete photo and files
POST	/admin/categories	Create category
PUT	/admin/categories/{id}	Update category
DELETE	/admin/categories/{id}	Delete category (if empty)
PUT	/admin/photos/reorder	Batch update sort_order
PUT	/admin/settings/{key}	Update site setting
GET	/admin/contact-submissions	List contact form submissions

Method	Endpoint	Description
PUT	/admin/contact-submissions/{id}/read	Mark as read

5.2 Image Processing Pipeline

On photo upload, the Spring Boot backend performs the following steps synchronously (async processing is a v2 optimization):

1. Validate file type (JPEG, HEIC, PNG) and size (max 30MB)
2. Extract EXIF metadata (using Apache Commons Imaging or metadata-extractor)
3. Auto-rotate based on EXIF orientation
4. Generate three sizes: thumbnail (400px), medium (1200px), full (2400px longest edge)
5. Convert to WebP with JPEG fallback
6. Generate tiny blur-up placeholder (20px wide, base64-encoded)
7. Store files to image volume, save metadata to PostgreSQL

6. Frontend Architecture (Angular)

6.1 Module Structure

Using Angular 19 standalone components throughout (no NgModules). Lazy-loaded route-based code splitting for performance.

```
src/app/
  core/            → auth interceptor, guards, API service
  shared/          → lightbox, image-loader, lazy-image components
  features/
    gallery/       → gallery-grid, gallery-detail, category-filter
    about/         → about-page component
    contact/       → contact-form component
    admin/         → login, photo-manager, category-manager, settings
  layouts/         → public-layout, admin-layout (shell components)
```

6.2 Key Angular Patterns

- Signals for reactive state management (Angular 19 signals API)
- Functional guards and resolvers for route protection and data prefetching
- HttpClient with interceptors for JWT attachment and error handling
- @defer blocks for lazy loading heavy gallery components
- Intersection Observer directive for lazy image loading
- Angular CDK DragDrop for admin photo reordering

7. Backend Architecture (Spring Boot)

7.1 Project Structure

```
src/main/java/com/lenslife/

  config/          → SecurityConfig, CorsConfig, WebMvcConfig

  controller/      → REST controllers (Gallery, Admin, Auth, Contact)

  service/         → Business logic (PhotoService, ImageProcessingService)

  repository/      → JPA repositories

  entity/          → JPA entities (Photo, Category, ContactSubmission)

  dto/             → Request/Response DTOs

  security/        → JwtTokenProvider, JwtAuthFilter

  exception/       → GlobalExceptionHandler, custom exceptions
```

7.2 Key Spring Boot Dependencies

Dependency	Purpose
spring-boot-starter-web	REST API framework
spring-boot-starter-data-jpa	Database ORM (Hibernate)
spring-boot-starter-security	Authentication & authorization
spring-boot-starter-validation	Request validation (Jakarta)
jjwt (io.jsonwebtoken)	JWT token generation and parsing
metadata-extractor	EXIF data extraction from photos
imgscalr / Thumbnailator	Image resizing and format conversion
flyway-core	Database migration versioning
postgresql (driver)	PostgreSQL JDBC driver
spring-boot-starter-mail	SMTP email for contact form forwarding

8. Docker Configuration

8.1 docker-compose.yml Structure

Since learning Docker is a core objective, this section provides detailed guidance on the compose configuration. The compose file defines three services, two volumes, and one custom network.

Services

Service	Build Context	Ports	Depends On	Notes
nginx	./docker/nginx	80:80, 443:443	api	Serves Angular build, proxies /api to Spring Boot
api	./	8080:8080 (dev only)	db	Multi-stage Dockerfile: build with Maven, run with JRE
db	postgres:16-alpine	5432:5432 (dev only)	—	Init scripts in /docker-entrypoint-initdb.d/

Volumes

- pg_data — Persistent PostgreSQL data directory
- photo_storage — Uploaded and processed images, shared between api and nginx containers

8.2 Multi-Stage Dockerfiles

The Spring Boot Dockerfile uses a multi-stage build: Stage 1 builds the JAR with Maven, Stage 2 runs it on a minimal JRE alpine image. The Nginx Dockerfile builds the Angular app with Node, then copies the dist output into the Nginx image. This keeps production images small and secure.

8.3 Docker Learning Checkpoints

Key Docker concepts you'll work through as part of this project:

Concept	Where You'll Use It
Dockerfile basics (FROM, COPY, RUN, CMD)	Building each service image
Multi-stage builds	Spring Boot JAR build + JRE runtime
Docker Compose services & depends_on	Orchestrating all three containers
Named volumes vs bind mounts	pg_data (named) vs source code (bind mount in dev)
Networking (custom bridge network)	Inter-container communication (api → db)
Environment variables & .env files	Database credentials, JWT secrets, SMTP config
Health checks	DB readiness check before API starts
Docker build cache optimization	Layer ordering for fast rebuilds
Docker logs & debugging	Troubleshooting container issues

9. Hosting & Deployment

9.1 Recommended Hosting Options

Since this is a personal portfolio (not a high-traffic commercial site), cost efficiency matters. Here are the recommended options ranked by fit for this project:

Option	Monthly Cost	Docker Support	Best For	Tradeoffs
DigitalOcean Droplet	$6–12/mo	Full (your own VM)	Best Docker learning, full control	You manage the server
Railway	$5–20/mo	Native container deploy	Fastest to deploy, good DX	Less Docker learning, usage-based pricing
Hetzner VPS	$4–8/mo	Full (your own VM)	Best price-to-performance ratio	EU-based, you manage the server
AWS Lightsail	$7–14/mo	Full (your own VM)	AWS ecosystem familiarity	AWS complexity for a simple site
Fly.io	$0–10/mo	Native container deploy	Free tier, global edge	Less traditional Docker experience

9.2 Recommended Approach: DigitalOcean Droplet

For a project where learning Docker is a key goal, a VPS (like DigitalOcean or Hetzner) is the best choice. You get a real Linux server where you'll SSH in, install Docker, run docker compose up, set up SSL, and manage the full lifecycle. This gives you the most transferable Docker skills.

Deployment Workflow

8. Push code to GitHub repository
9. SSH into the Droplet
10. git pull && docker compose up -d --build
11. Nginx handles SSL via Let's Encrypt (certbot in a sidecar container or host-level)

Future improvement: Add a simple GitHub Actions CI/CD pipeline that builds, tests, and deploys on push to main.

9.3 Domain & DNS

Register a domain through Namecheap, Google Domains, or Cloudflare Registrar. Point A record to your Droplet's IP. Optionally use Cloudflare as a CDN/DNS proxy for caching and DDoS protection (free tier is sufficient).

10. Phased Delivery Roadmap

The project is broken into four phases. Each phase produces a deployable increment.

Phase 1: Foundation (Weeks 1–2)

Task	Details
Figma → Design tokens	Extract colors, typography, spacing from your Figma design
Docker Compose setup	Three-container setup running locally with hot reload
Spring Boot scaffold	Project init, Flyway migrations, health check endpoint
PostgreSQL schema	Run initial migration with all four tables
Angular scaffold	CLI project, routing skeleton, core service layer
Angular → Figma implementation	Translate Figma design to Angular components (layout shell, nav)

Phase 2: Gallery Core (Weeks 3–4)

Task	Details
Image upload API	POST endpoint with validation, EXIF extraction, resizing pipeline
Image serving	GET endpoint with size parameter, cache headers, WebP content negotiation
Admin photo management	Upload UI, metadata editing, category assignment
Public gallery grid	Responsive layout matching Figma, lazy loading, blur-up placeholders
Lightbox component	Full-screen view, swipe, EXIF overlay, keyboard navigation
Category filtering	Route-based filtering with animated transitions

Phase 3: Pages & Polish (Week 5)

Task	Details
About page	Admin-editable content, hero image, social links
Contact form	Frontend validation, API endpoint, honeypot, rate limiting
Admin dashboard	Contact submissions viewer, site settings editor
Performance pass	Lighthouse audit, image optimization tuning, bundle analysis
Responsive QA	Test across mobile, tablet, desktop breakpoints

Phase 4: Deploy & Launch (Week 6)

Task	Details
Production Docker config	Optimized images, resource limits, logging
Server provisioning	DigitalOcean Droplet setup, Docker install, firewall
SSL & domain setup	Let's Encrypt via certbot, DNS configuration
Deployment automation	Script or GitHub Actions for push-to-deploy
Monitoring	Basic uptime monitoring (UptimeRobot free tier)
Launch	Upload portfolio photos, go live

11. Future Considerations (v2+)

- Blog / journal section (rich text editor, MDX support)
- Cloud image storage migration (AWS S3 or Cloudflare R2)
- Async image processing with message queue (RabbitMQ or Spring Events)
- Print sales integration (Stripe checkout)
- Photo tagging system with search
- Client proofing galleries (password-protected albums)
- Progressive Web App (PWA) support
- Analytics dashboard (view counts, popular photos)
- Kubernetes migration for Docker learning progression

End of Document